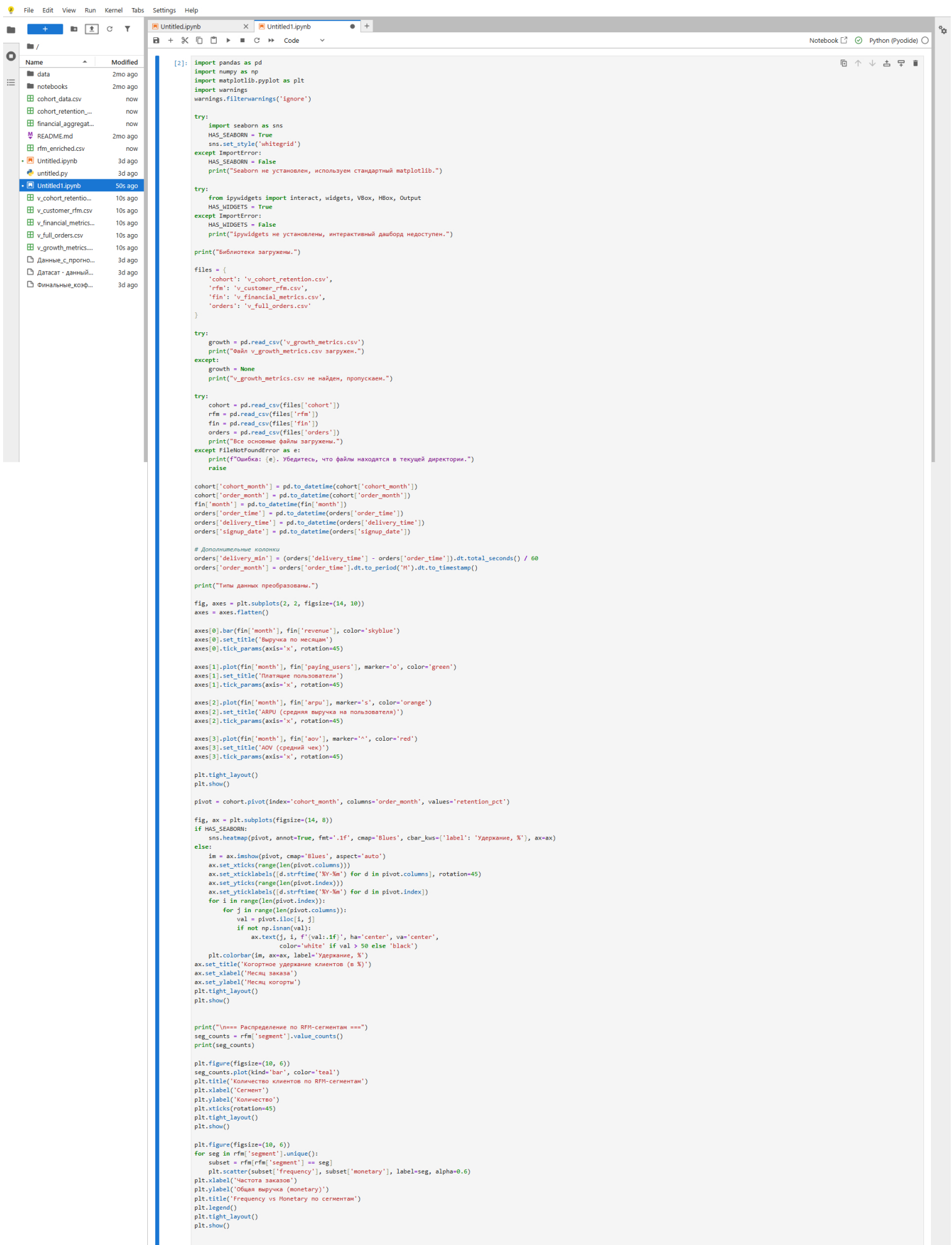




```

status_counts = orders['status'].value_counts()
plt.figure(figsize=(8, 6))
status_counts.plot(kind='pie', autopct='%1.1f%%', startangle=90)
plt.title('Статус заказов')
plt.ylabel('')
plt.tight_layout()
plt.show()

top_cuisines = orders['cuisine'].value_counts().head(10)
plt.figure(figsize=(10, 6))
top_cuisines.plot(kind='barh', color='coral')
plt.title('Топ-10 кухонь по количеству заказов')
plt.xlabel('Количество заказов')
plt.tight_layout()
plt.show()

city_rating = orders.groupby('city')['rating'].mean().sort_values(ascending=False).head(10)
plt.figure(figsize=(10, 6))
city_rating.plot(kind='bar', color='purple')
plt.title('Средний рейтинг по городам (top-10)')
plt.xlabel('Город')
plt.ylabel('Средний рейтинг')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

plt.figure(figsize=(10, 6))
scatter = plt.scatter(orders['rating'], orders['delivery_min'],
                      c=orders['status'].astype('category').cat.codes,
                      alpha=0.6, cmap='viridis')
plt.colorbar(scatter, label='Код статуса')
plt.xlabel('Рейтинг ресторана')
plt.ylabel('Время доставки (мин)')
plt.title('Время доставки vs Рейтинг')
plt.tight_layout()
plt.show()

if HAS_WIDGETS:
    date_start = widgets.DatePicker(description='C', value=fin['month'].min())
    date_end = widgets.DatePicker(description='По', value=fin['month'].max())
    segment_dropdown = widgets.Dropdown(
        options=['Bce'] + sorted(rfm['segment'].unique()),
        value='Bce',
        description='Сегмент:'
    )
    output = Output()

    def update_dashboard(month_start, month_end, segment):
        with output:
            clear_output(wait=True)

            mask_fin = (fin['month'] >= pd.to_datetime(month_start)) & (fin['month'] <= pd.to_datetime(month_end))
            fin_filtered = fin[mask_fin]

            if segment != 'Bce':
                rfm_filtered = rfm[rfm['segment'] == segment]
            else:
                rfm_filtered = rfm

            fig1, ax1 = plt.subplots(figsize=(12, 5))
            ax1.bar(fin_filtered['month'], fin_filtered['revenue'], color='blue', alpha=0.6, label='Выручка')
            ax1.set_xlabel('Месяц')
            ax1.set_ylabel('Выручка', color='blue')
            ax1.tick_params(axis='x', rotation=45)
            ax2 = ax1.twinx()
            ax2.plot(fin_filtered['month'], fin_filtered['arpu'], color='red', markers='o', label='ARPU')
            ax2.set_ylabel('ARPU', color='red')
            plt.title(f'Выручка и ARPU (сегмент: {segment})')
            fig1.legend(loc='upper left')
            plt.tight_layout()
            plt.show()

            if segment == 'Bce':
                fig2, ax2 = plt.subplots(figsize=(8, 6))
                rfm_filtered['frequency'].plot(kind='bar', ax=ax2, color='teal')
                ax2.set_title('Все клиенты по сегментам')
                ax2.set_xlabel('Сегмент')
                ax2.set_ylabel('Количество')
                plt.tight_layout()
                plt.show()
            else:
                fig2, ax2 = plt.subplots(figsize=(8, 6))
                rfm_filtered['frequency'].hist(bins=10, ax=ax2, color='orange', edgecolor='black')
                ax2.set_title(f'Распределение частоты заказов в сегменте {segment}')
                ax2.set_xlabel('Частота')
                ax2.set_ylabel('Количество клиентов')
                plt.tight_layout()
                plt.show()

            fig3, ax3 = plt.subplots(figsize=(10, 6))
            for seg in rfm_filtered['segment'].unique():
                subset = rfm_filtered[rfm_filtered['segment'] == seg]
                ax3.scatter(subset['frequency'], subset['monetary'], label=seg, alpha=0.6)
            ax3.set_xlabel('Частота')
            ax3.set_ylabel('Monetary')
            ax3.set_title(f'Frequency vs Monetary (сегмент: {segment})')
            ax3.legend()
            plt.tight_layout()
            plt.show()

            print(f"Всего клиентов в выбранном сегменте: {len(rfm_filtered)}")
            if segment != 'Bce':
                print(f"Средняя частота: {rfm_filtered['frequency'].mean():.2f}")
                print(f"Средний monetary: {rfm_filtered['monetary'].mean():.2f}")

        interact(update_dashboard, month_start=date_start, month_end=date_end, segment=segment_dropdown)
        display(VBox([HBox([date_start, date_end, segment_dropdown]), output]))
    else:
        print("\nИнтерактивный дашборд недоступен. Установите ipywidgets (pip install ipywidgets).")

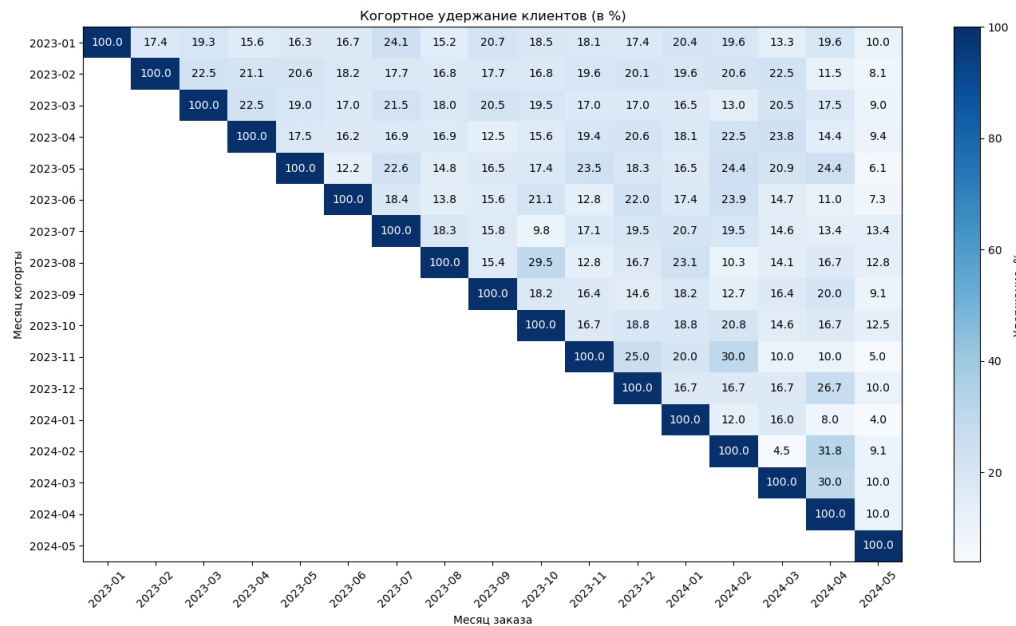
fin.to_csv('financial_aggregated.csv', index=False)
rfm.to_csv('rfm_enriched.csv', index=False)
cohort.to_csv('cohort_data.csv', index=False)

pivot_retention = cohort.pivot(index='cohort_month', columns='order_month', values='retention_pct')
pivot_retention.to_csv('cohort_retention_pivot.csv')

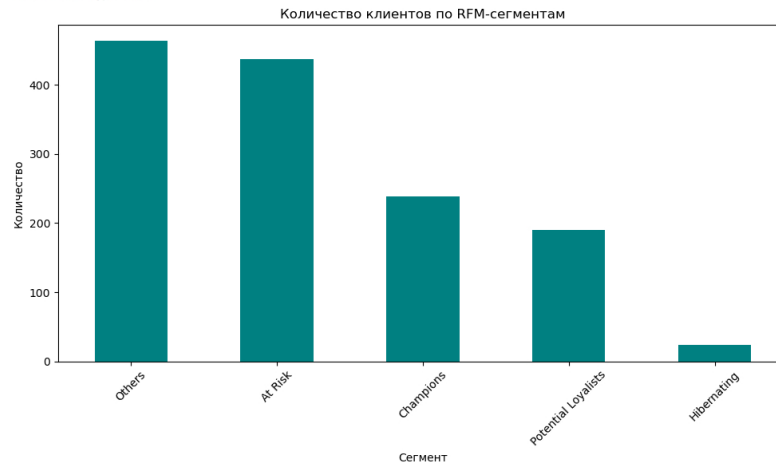
print("\nЭкспорт завершен. Созданы файлы:")
print("- financial_aggregated.csv")
print("- rfm_enriched.csv")
print("- cohort_data.csv")
print("- cohort_retention_pivot.csv")
print("\nАнализ успешно выполнен.")

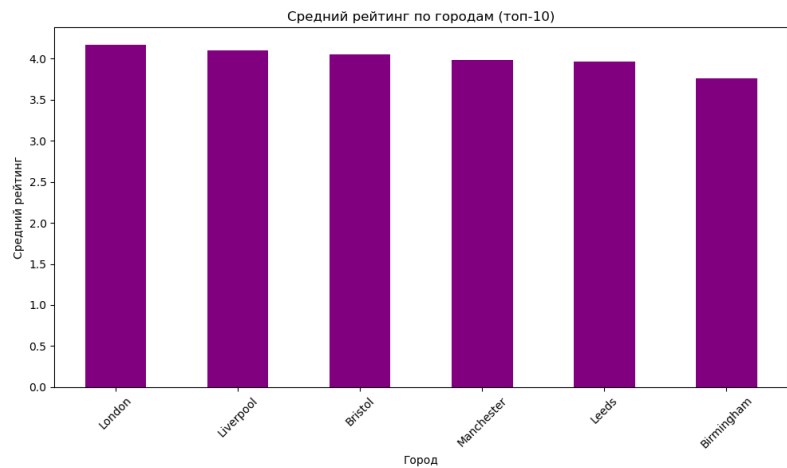
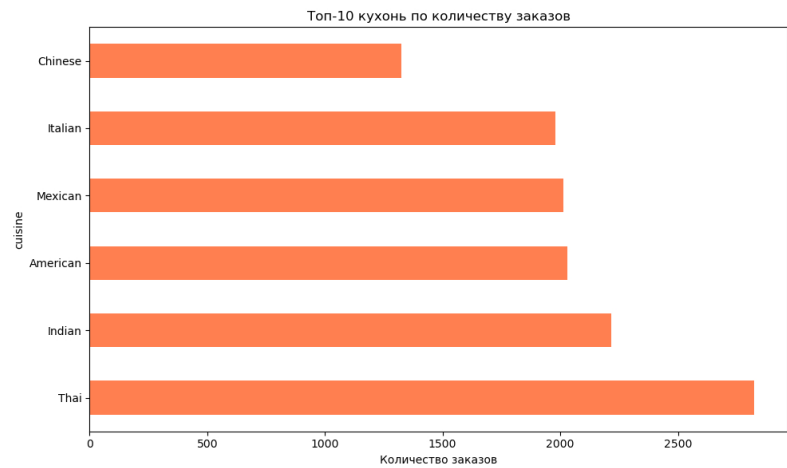
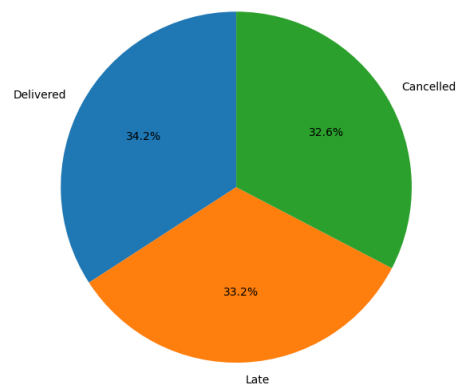
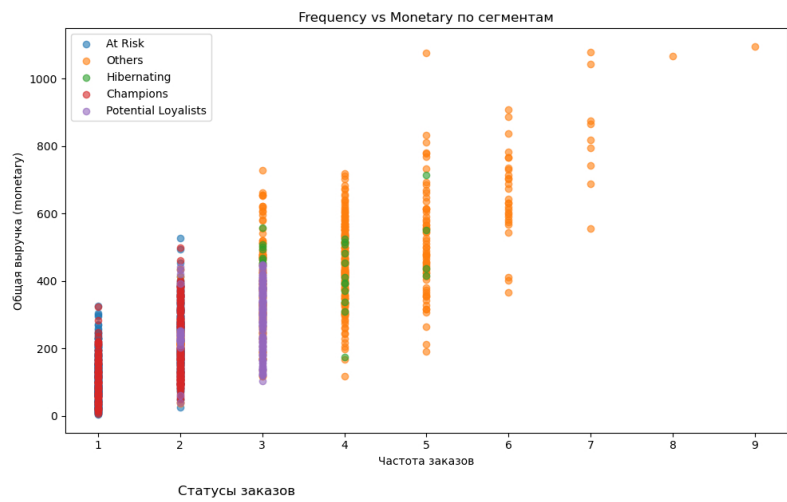
Seaborn не установлен, используем стандартный matplotlib.
ipywidgets не установлены, интерактивный дашборд недоступен.
Библиотеки загружены.
Файл v_growth_metrics.csv загружен.
Все основные файлы загружены.
Типы данных преобразованы.

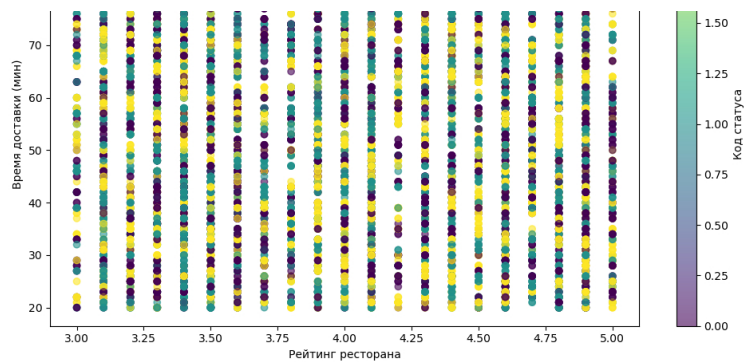
```



=== Распределение по RFM-сегментам ===  
segment  
Others 463  
At Risk 437  
Champions 239  
Potential Loyalists 190  
Hibernating 24  
Name: count, dtype: int64







Интерактивный дашборд недоступен. Установите ipywidgets (pip install ipywidgets).

Экспорт завершен. Созданы файлы:

- financial\_aggregated.csv
- rfm\_enriched.csv
- cohort\_data.csv
- cohort\_retention\_pivot.csv

Анализ успешно выполнен.